Step 13 - Access Control and Traps 1
Reference Note 1
Learning Objectives 1
1. Why Access Control Is Needed 1
2. Access Control Hierarchy 2
3. What Is a Trap? 2
4. Why Trap Timer Access? 3
5. CNTKCTL_EL1 3
Simplified Role 4
Conceptual View 4
6. CNTHCTL_EL2 4
Simplified Role 4
Conceptual View 4
7. Access Control Flow 5
Normal Access 5
Trapped Access 5
8. Example: Guest Access 5
9. Who Controls What? 6
Responsibility Table 6
10. Relationship to Virtualization 6
Architecture Summary 7
Common Misconceptions 7
Misconception 1 7
Misconception 2 7
Misconception 3 8
Key Takeaways 8
Knowledge Check 8
Q1 8
Q2 8

Q3 8
Q4 9
Q5 9
Answer Key 9

Step 13 - Access Control and Traps

Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture and ARMv8-A architecture documentation.

Previous chapters introduced:

- Physical Counter (`CNTPCT_EL0`)
- Virtual Counter (`CNTVCT_EL0`)
- Physical Timer
- Virtual Timer
- `CNTVOFF_EL2`

This chapter explains:

Who is allowed to access timer registers?
Who controls that access?
What happens when access is denied?

Learning Objectives

After completing this chapter, you should understand:

- Timer access permissions
- Timer ownership across exception levels
- CNTKCTL_EL1

- CNTHCTL_EL2
- Timer access trapping
- Why hypervisors intercept timer operations
- Relationship between permissions and virtualization

1. Why Access Control Is Needed

Suppose an application running at:

EL0

tries to read:

MRS X0, CNTPCT_EL0

Should ARM allow this?

Possible answers:

Policy	Result
Allow	Application reads timer directly
Deny	Trap to higher exception level

ARM allows the system software to decide.

2. Access Control Hierarchy

Exception Level	Typical Role
EL3	Secure Monitor
EL2	Hypervisor
EL1	Operating System
EL0	Application

Higher exception levels may control timer access for lower levels.

3. What Is a Trap?

A trap occurs when:

Software attempts an operation
↓
Architecture redirects control
↓
Higher exception level handles request

Example:

EL0 reads timer

Instead of succeeding:

Exception Generated

Control goes to:

EL1

4. Why Trap Timer Access?

Virtualization often requires control over:

Time
Timers
Interrupts

The hypervisor may want to:

Reason	Purpose
Hide physical time	Guest isolation
Emulate timer behavior	Virtual machine support
Monitor accesses	Security and control
Implement policies	Resource management

5. CNTKCTL_EL1

`CNTKCTL_EL1` controls timer visibility to EL0.

Think of it as:

OS Timer Access Control Register

Simplified Role

Operation	Controlled By
EL0 reads counter	CNTKCTL_EL1
EL0 reads timer registers	CNTKCTL_EL1
EL0 event stream access	CNTKCTL_EL1

Conceptual View

Setting	Result
Enable Access	EL0 reads timer registers
Disable Access	Access traps or is blocked

6. CNTHCTL_EL2

`CNTHCTL_EL2` controls timer access from EL1.

Think of it as:

Hypervisor Timer Access Control Register

Simplified Role

Access Attempt	Controlled By

EL1 reads physical counter	CNTHCTL_EL2
EL1 accesses timer resources	CNTHCTL_EL2
EL1 virtualization behavior	CNTHCTL_EL2

Conceptual View

Setting	Result
Allow Access	EL1 accesses timer directly
Trap Access	EL2 intercepts request

7. Access Control Flow

Normal Access

Step	Action
1	EL1 reads CNTPCT_EL0
2	Permission check
3	Access allowed
4	Counter value returned

Trapped Access

Step	Action
1	EL1 reads CNTPCT_EL0
2	Permission check
3	Trap generated
4	Control transfers to EL2

8. Example: Guest Access

Suppose:

Guest OS at EL1

attempts:

MRS X0, CNTPCT_EL0

Hypervisor may choose:

Option	Result
Allow	Guest receives physical counter
Trap	EL2 intercepts operation

This enables timer virtualization.

9. Who Controls What?

Register	Owner
CNTKCTL_EL1	EL1
CNTHCTL_EL2	EL2
CNTVOFF_EL2	EL2

Responsibility Table

Level	Controls
EL1	EL0 timer access
EL2	EL1 timer access
EL3	System-wide secure policy

10. Relationship to Virtualization

Virtualization requires:

Guest Time
Guest Timers
Guest Interrupts

Without trapping:

Guest accesses physical resources directly

With trapping:

Hypervisor controls what guest sees

This is one of the fundamental mechanisms behind timer virtualization.

Architecture Summary

Component	Responsibility
CNTKCTL_EL1	Controls EL0 timer access
CNTHCTL_EL2	Controls EL1 timer access
Trap Mechanism	Redirects access to higher EL
Hypervisor	Manages guest timer visibility
Guest	Uses approved timer resources

Common Misconceptions

Misconception 1

CNTPCT_EL0 means only EL0 can use it

False.

The register name does not define ownership.

Higher exception levels can also access it.

Misconception 2

EL1 always accesses timer registers directly

False.

EL2 can trap EL1 accesses.

Misconception 3

Traps indicate an error

False.

Traps are often intentional and are fundamental to virtualization.

Key Takeaways

Concept	Key Point
CNTKCTL_EL1	Controls EL0 timer access
CNTHCTL_EL2	Controls EL1 timer access
Trap	Redirect access to higher EL
EL1	Controls applications
EL2	Controls guests
Virtualization	Relies heavily on controlled timer access

Knowledge Check

Q1

What is a timer trap?

Q2

Which register controls EL0 timer access?

Q3

Which register controls EL1 timer access?

Q4

Why would a hypervisor trap timer accesses?

Q5

Does `_EL0` in `CNTPCT_EL0` mean only EL0 can access it?

Answer Key

Question	Answer
Q1	Redirection of timer access to a higher exception level
Q2	CNTKCTL_EL1
Q3	CNTHCTL_EL2
Q4	To control or virtualize timer behavior
Q5	No